

Battle Peach：微服務擂台

來源：<https://codelabs.developers.google.com/codelabs/battle-peach?hl=zh-tw>

步驟數：3

在本程式碼研究室中，您將建構一個微服務，透過在競技場中「丟擲」顆粒來對抗其他微服務。

目錄

1. [簡介](#)
2. [部署微服務](#)
3. [恭喜](#)

步驟 1 · 約 0 分鐘

1. 簡介

上次更新時間：2020 年 2 月 12 日

微服務戰鬥競技場

你是否曾參與雪仗，在移動的同時，向其他人丟雪球？如果還沒用過，不妨找機會試試！但現在您不必擔心被揍，可以建構小型網路存取服務 (微服務)，與其他微服務展開史詩般的戰鬥。由於我們在喬治亞州亞特蘭大舉辦第一場微服務大戰，因此微服務會投擲桃子，而不是雪球。

你可能想知道... 但微服務如何「將桃子丟向」其他微服務？微服務可以接收網路要求 (通常是透過 HTTP)，並傳回回應。系統會提供「競技場管理員」，將競技場的目前狀態傳送給微服務，然後微服務會傳回指令，指定要執行的動作。

當然，目標是贏得勝利，但過程中您會瞭解如何在 Google Cloud 上建構及部署微服務。

運作方式

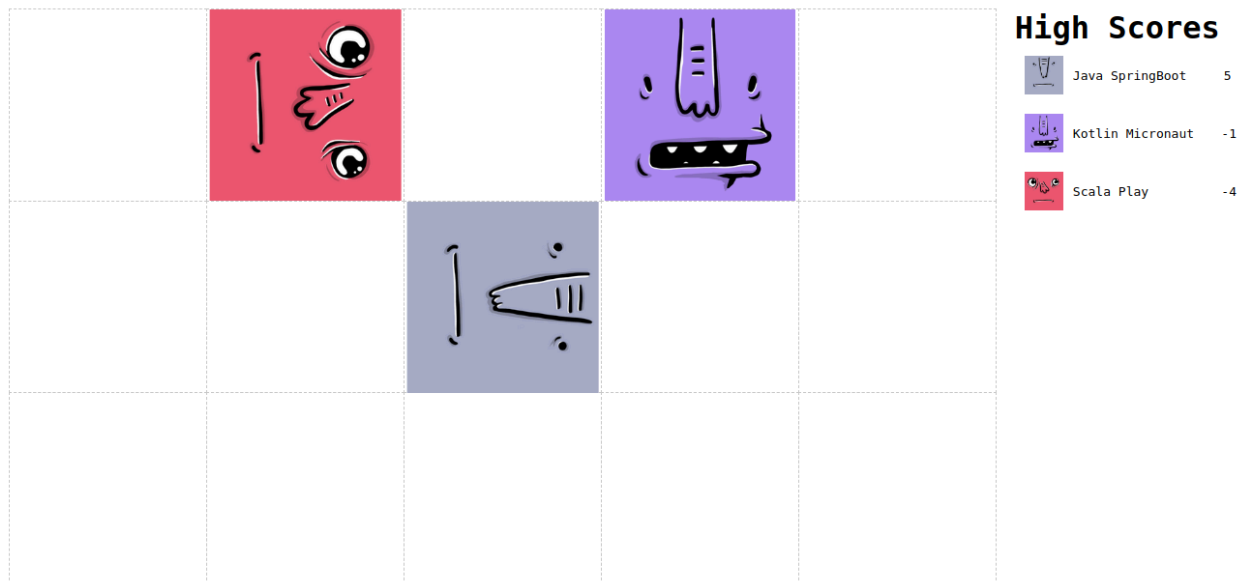
您可以使用任何技術建構微服務 (或從 Java、Kotlin 或 Scala 啟動器中選擇)，然後在 Google Cloud 上部署微服務。部署完成後，請填寫表單，告知我們微服務的網址，我們會將其加入競技場。

競技場包含特定戰鬥的所有玩家。DevNexus 會議每天都會有一個競技場。每個玩家都代表一項微服務，會四處移動並向其他玩家投擲桃子。

競技場管理員大約每秒會呼叫一次微服務，傳送目前的競技場狀態 (玩家所在位置)，而微服務會回覆要執行的指令。在競技場中，您可以向前移動、向左或向右轉，或是投擲桃子。投出的桃子會朝玩家面向的方向移動，最多可移動三格。如果桃子「擊中」其他玩家，投擲者可得一分，被擊中的玩家則會扣一分。系統會根據目前的玩家人數自動調整競技場大小。

以下是競技場的樣子，其中有三位虛構玩家：

devnexus



Battle Peach 競技場示例

循環衝突

在競技場中，多位玩家可能會嘗試執行衝突的動作。舉例來說，兩位玩家可能會嘗試移動到同一個空間。如有衝突，回應時間最短的微服務會勝出。

觀看對戰

如要查看微服務在戰鬥中的表現，請[前往即時競技場](#)！

Battle API

如要與競技場管理員合作，微服務必須實作特定 API，才能參與競技場。競技場管理員會透過 HTTP POST 將目前的競技場狀態傳送至您提供的網址，JSON 結構如下：

□□□□

```
{
  "_links": {
    "self": {
      "href": "https://YOUR_SERVICE_URL"
    }
  },
  "arena": {
    "dims": [4,3], // width, height
    "state": {
      "https://A_PLAYERS_URL": {
        "x": 0, // zero-based x position, where 0 = left
        "y": 0, // zero-based y position, where 0 = top
        "direction": "N", // N = North, W = West, S = South, E = East
        "wasHit": false,
        "score": 0
      }
      ... // also you and the other players
    }
  }
}
```

HTTP 回應必須是狀態碼 200 (OK)，且回應內文包含您的下一步行動，並編碼為下列其中一個大寫字元：

```
F <- move Forward
R <- turn Right
L <- turn Left
T <- Throw
```

就是這麼簡單！接下來，我們將逐步說明如何在 [Cloud Run](#) 上部署微服務。Cloud Run 是 Google Cloud 服務，可執行微服務和其他應用程式。

2. 部署微服務

只要微服務可公開存取並符合 Battle API 規定，您就能使用任何技術建構微服務，並部署至任何位置。為求簡單，您可以從只選取隨機指令的範例專案開始。

選取要開始使用的範例

您可以從下列三個戰鬥微服務範例著手：

Java 和 Spring Boot	資料來源	在 Cloud Run 上部署
Java 和 Quarkus	資料來源	在 Cloud Run 上部署
Kotlin 和 Micronaut	資料來源	在 Cloud Run 上部署
Kotlin 和 Quarkus	資料來源	在 Cloud Run 上部署
Scala 和 Play Framework	資料來源	在 Cloud Run 上部署
Go	資料來源	在 Cloud Run 上部署

決定要從哪個範例開始後，請點選上方的「Deploy on Cloud Run」（部署至 Cloud Run）按鈕。系統會啟動 [Cloud Shell](#)（雲端虛擬機器的網頁版控制台），並在其中複製來源、建構可部署的套件（Docker 容器映像檔），然後上傳至 [Google Container Registry](#)，最後部署到 [Cloud Run](#)。

系統詢問時，請指定 `us-central1` 區域。

下方的螢幕截圖顯示微服務建構和部署的 *Cloud Shell* 輸出內容

```

t [PROJECT_ID]"
demo@cloudshell:~$ cloudshell_open --repo_url "https://github.com/jamesward/hello-netcat.git" --page "editor"
[ ✓ ] Cloned git repository https://github.com/jamesward/hello-netcat.git.
[ ✓ ] Queried list of your GCP projects
[ ? ] Would you like to use existing GCP project jw-demo to deploy this app? Yes
[ ✓ ] Enabled Cloud Run API on project jw-demo.
[ ? ] Choose a region to deploy this application: us-central1
[ ! ] Attempting to build this application with its Dockerfile...
[ ! ] FYI, running the following command:
      docker build -t gcr.io/jw-demo/hello-netcat .
[ ✓ ] Built container image gcr.io/jw-demo/hello-netcat
[ ! ] FYI, running the following command:
      docker push gcr.io/jw-demo/hello-netcat
[ ✓ ] Pushed container image to Google Container Registry.
[ ! ] FYI, running the following command:
      gcloud run deploy hello-netcat\
        --project=jw-demo\
        --platform=managed\
        --region=us-central1\
        --image=gcr.io/jw-demo/hello-netcat\
        --memory=512Mi\
        --allow-unauthenticated
[ ✓ ] Successfully deployed service hello-netcat to Cloud Run.
* This application is billed only when it's handling requests.
* Manage this application at Cloud Console:
  https://console.cloud.google.com/run?project=jw-demo
* Learn more about Cloud Run:
  https://cloud.google.com/run/docs
[ ✓ ] Your application is now live here:
  https://hello-netcat-weurlhfjmq-uc.a.run.app
demo@cloudshell:~$ █

```

確認微服務是否正常運作

在 Cloud Shell 中，您可以向新部署的微服務提出要求，並將 `YOUR_SERVICE_URL` 替換為服務的網址 (位於 Cloud Shell 中「Your application is now live here」這一行之後)：

```
curl -d '{
  "_links": {
    "self": {
      "href": "https://foo.com"
    }
  },
  "arena": {
    "dims": [4,3],
    "state": {
      "https://foo.com": {
        "x": 0,
        "y": 0,
        "direction": "N",
        "wasHit": false,
        "score": 0
      }
    }
  }
}' -H "Content-Type: application/json" -X POST -w "\n" \
https://YOUR_SERVICE_URL
```

您應該會看到 F、L、R 或 T 的回應字串。

要求納入競技場

如要加入競技場，請[填寫簡易表單](#)。最困難的部分是決定要使用哪張圖片做為個人資料相片。你可以使用 GitHub 圖片或 LinkedIn 圖片，也可以由我們隨機挑選顯示圖片。審查完畢後，你的球員就會出現在[競技場](#)。

進行及部署變更

如要進行變更，您必須在 Cloud Shell 中設定 GCP 專案和所用範例的相關資訊。首先列出您的 GCP 專案：

```
gcloud projects list
```

您可能只有一個專案。從第一欄複製 `PROJECT_ID`，然後貼到下列指令中 (將 `YOUR_PROJECT_ID` 替換為實際的專案 ID)，以便設定環境變數，供後續指令使用：

```
export PROJECT_ID=YOUR_PROJECT_ID
```

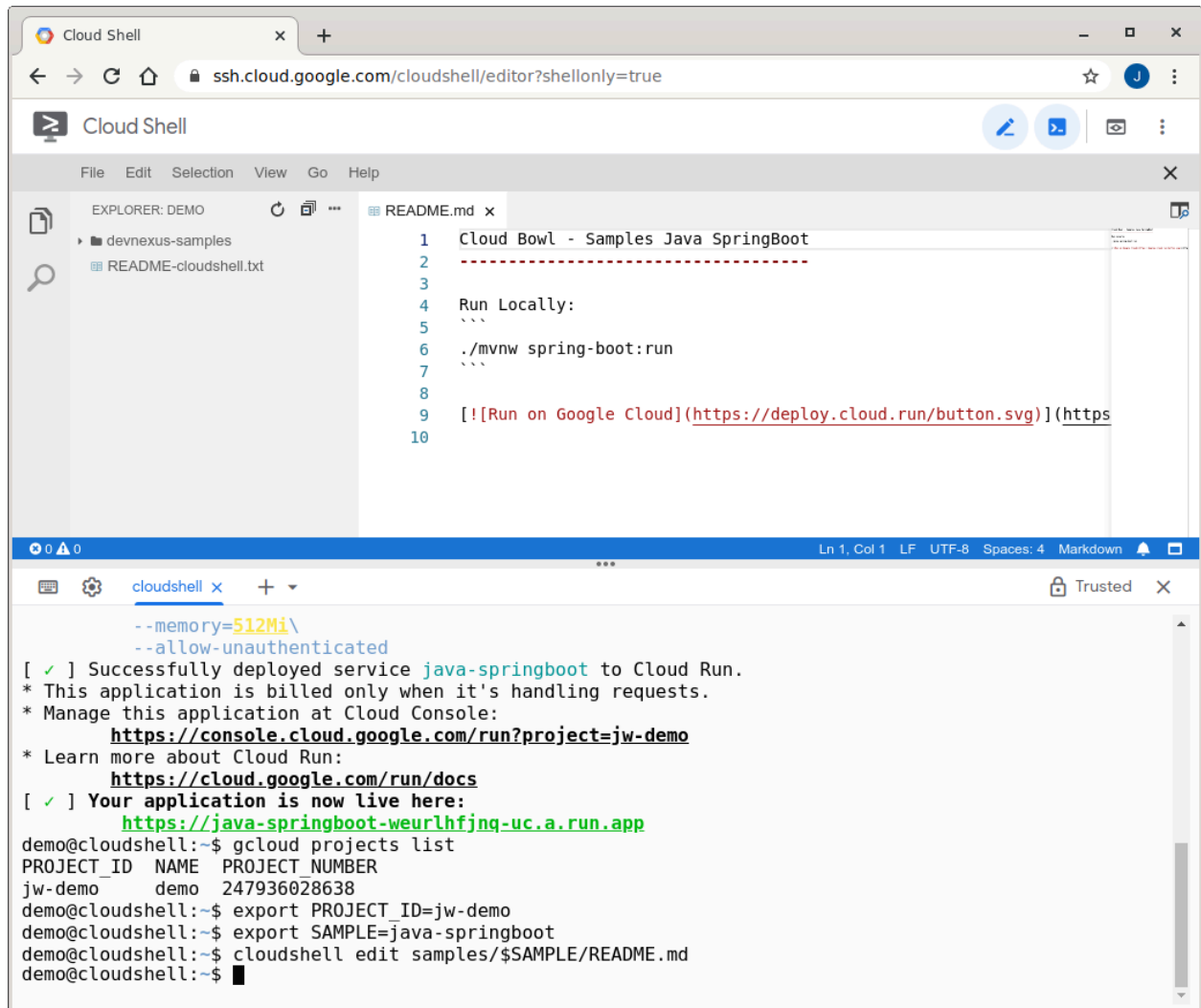
現在請為您使用的範例設定另一個環境變數，以便在後續指令中指定正確的目錄和服務名稱：

```
# Copy and paste ONLY ONE of these
export SAMPLE=java-springboot
export SAMPLE=kotlin-micronaut
export SAMPLE=scala-play
```

現在，您可以在 Cloud Shell 中編輯微服務的來源。如要開啟 Cloud Shell 網頁版編輯器，請執行下列指令：

```
cloudshell edit cloudbowl-microservice-game/samples/$SAMPLE/README.md
```

接著，系統會顯示變更的進一步操作說明。



Cloud Shell 編輯器已開啟範例專案

儲存變更後，請使用 `pack` 指令在 Cloud Shell 中建構專案。這項指令會使用建構套件偵測專案類型、編譯專案，並建立可部署的構件 (Docker 容器映像檔)。

```
pack build gcr.io/$PROJECT_ID/$SAMPLE \
  --path cloudbowl-microservice-game/samples/$SAMPLE \
  --builder heroku/buildpacks
```

容器映像檔建立完成後，請使用 `docker` 指令 (在 Cloud Shell 中) 將容器映像檔推送至 Google Container Registry，以便 Cloud Run 存取：

```
docker push gcr.io/$PROJECT_ID/$SAMPLE
```

現在將新版本部署至 Cloud Run：


```
gcloud run deploy $SAMPLE\  
  --project=$PROJECT_ID\  
  --platform=managed\  
  --region=us-central1\  
  --image=gcr.io/$PROJECT_ID/$SAMPLE\  
  --memory=512Mi\  
  --allow-unauthenticated
```

現在競技場就會使用新版本！

步驟 3 · 約 0 分鐘

3. 恭喜

恭喜！您已成功建構及部署微服務，可與其他微服務對戰！祝您好運！

後續步驟

- [在 Spring Boot 應用程式中新增分散式追蹤](#)
- [使用 AI Platform 讓微服務變得超級智慧](#)

參考文件

- [Cloud Run 說明文件](#)
-